LI.FI

LI.FI Intent: AxelarOracle + ChainlinkCCI-POracle

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1 About Researcher **1**

2 Disclaimer **1**

3 Scope **1**

4 Risk classification **1**
- 4.1 Impact . . . 2
- 4.2 Likelihood . . . 2
- 4.3 Action required for severity levels . . . 2

5 Executive Summary **2**

6 Findings **4**
- 6.1 Low Risk . . . 4
  - 6.1.1 AxelarOracle accepts chain names that Axelar rejects and commits their chain IDs permanently 4
- 6.2 Informational . . . 5
  - 6.2.1 The vendored receiver interface names the OffRampRouter as msg.sender even though the Router calls the receiver . . . 5
  - 6.2.2 The default CCIP destination gas is not sized to the submitted payload batch . . . 5

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ethereum, Ripple, Uniswap, Morpho, Injective, Phantom, Cosmos and received bug-bounties from those projects.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

AxelarOracle	`src/integrations/oracles/axelar/AxelarOracle.sol`
AxelarStrings	`src/integrations/oracles/axelar/AxelarStrings.sol`
SolanaEnvelope	`src/integrations/oracles/axelar/SolanaEnvelope.sol`
MessageEncodingLib	`src/libs/MessageEncodingLib.sol`
ChainMap	`src/oracles/ChainMap.sol`
ChainlinkCCIPOracle	`src/integrations/oracles/chainlink/ChainlinkCCIPOracle.sol`
Chainlink/external/*	`src/integrations/oracles/chainlink/external/*`

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 17 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document (”scope”).

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **3 issues** were found.

Project Summary

Project Name	**Lifi Intent**
Project Type	Solidity Smart Contracts
Repository	catalystsystem/lifi-intent
Commit 1	e57f0b261
Commit 2	b382bb12
Audit Timeline	October 20, 2026 - October 29, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	**0**
High Risk	**0**
Medium Risk	**0**
Low Risk	**1**
Gas Optimizations	**0**
Informational	**2**
Total Issues	**3**

6 Findings

6.1 Low Risk

6.1.1 AxelarOracle accepts chain names that Axelar rejects and commits their chain IDs permanently

CONTEXT: AxelarOracle.sol#L79-L81

DESCRIPTION: `setChainMap` checks that a key round-trips through the local packing functions. This admits every lowercase or non-uppercase, NUL-free name from 1 to 32 bytes. Axelar's `ChainNameRaw` accepts only ASCII names of at most 20 bytes and rejects `_`. The local rule therefore accepts names that the Axelar network cannot verify or route, including the underscore, 32-byte, and UTF-8 values exercised by `test/oracle/axelar/AxelarOracle.t.sol::testSetChainMapAcceptsAnyCharsetAndFullWord`.

`ChainMap` writes the forward and reverse entries together and permits neither key nor chain ID to be used again. If the owner maps one of these names, or a valid-looking typo such as `stelar`, to the intended OIF chain ID, that chain ID is permanently bound to the bad name. Outbound calls use the bad name and cannot be routed. Inbound messages from the real chain use a different key, and the correct name cannot later be assigned to the occupied chain ID. Supporting the route then requires a new oracle deployment. A bad outbound submission can also pay the Axelar gas service before its destination-name error is discovered.

Only the owner can create this state, and source attestations are not consumed by `submit`, so a third party cannot use it to forge a proof or disable an existing correctly configured route.

POC:

Add this test to the existing `AxelarOracleTest` contract:

```
function testInvalidAxelarChainNamePermanentlyConsumesChainId() public {
    uint256 chainId = 2;
    uint256 invalidName = pack("e_vm.2");
    uint256 validName = pack("ethereum");

    axelar.setChainMap(invalidName, chainId);

    assertEq(axelar.chainIdMap(invalidName), chainId);
    assertEq(axelar.reverseChainIdMap(chainId), invalidName);

    vm.expectRevert(ChainMap.AlreadySet.selector);
    axelar.setChainMap(validName, chainId);
}
```

RECOMMENDATION: Apply Axelar's current chain-name grammar in `setChainMap`: at most 20 ASCII bytes and no `_`, in addition to the existing lowercase, packing, and NUL checks. Generate configuration transactions from a reviewed Axelar chain registry that binds each name to its OIF chain ID; syntax checks alone cannot catch a valid-looking typo. Apply the same validation in `ConfigureAxelarChainMap` and add rejection tests for underscores, non-ASCII bytes, and names longer than 20 bytes.

LI.FI: Fixed in 8e3efc7.

RESEARCHER: Verified fix.

6.2 Informational

6.2.1 The vendored receiver interface names the OffRampRouter as msg.sender even though the Router calls the receiver

CONTEXT: IAny2EVMMessageReceiver.sol#L15-L17

DESCRIPTION: The vendored interface says it differs from Chainlink CCIP `contracts-ccip-v2.0.0` only in formatting, but its security note changes the expected caller from the `Router` to the `OffRampRouter`. At the pinned upstream commit, the OffRamp invokes `Router.routeMessage`, and the Router makes the external `ccipReceive` call. `CCIPReceiver.onlyRouter` implements the correct check, so the audited oracle's runtime behavior is unaffected.

The modified note makes the provenance statement inaccurate and gives the wrong guidance to future contracts that reuse the interface. An implementation that follows it and checks the OffRamp address would reject every normal delivery.

RECOMMENDATION: Restore the upstream wording so the note names the `Router`. If the wording is intentionally retained, list it as a modification and explain the actual OffRamp-to-Router-to-receiver call path.

LI.FI: Fixed in 254835c. The security note now names the Router as msg.sender, matching the upstream file at c2c125c, so the "Modifications: none besides formatting" header is accurate again.

RESEARCHER: Verified fix.

6.2.2 The default CCIP destination gas is not sized to the submitted payload batch

CONTEXT: ChainlinkCCIPOracle.sol#L118-L120

DESCRIPTION: `_buildMessage` forwards `extraArgs` unchanged. Empty extra arguments use the destination lane's default callback limit, documented as 200,000 gas for EVM messages in the vendored `Client` library. That default is independent of both the number and size of the encoded payloads.

The destination callback hashes every payload, writes a fresh storage slot, and emits an event. Its cost therefore grows with the batch. On the audited build, direct receiver calls with distinct 256-byte payloads used about 28,600 gas for one payload, 179,700 for seven, and 204,900 for eight. A lane using the documented 200,000 gas default can consequently mark an eight-payload delivery `FAILED`. The message remains available for permissionless manual execution with a larger gas override, but the original CCIP fee is not returned and the proof is delayed until someone performs that retry. Each submitter controls its own `extraArgs`, so this does not let a third party interfere with another submission.

RECOMMENDATION: Document that the protocol default is not a batch-size estimate. The calling SDK or submission service should set a destination-specific gas or compute limit from the encoded message size and payload count, with a margin for protocol and compiler changes. Test the largest supported batch on each destination family and document CCIP's manual-execution recovery path.

LI.FI: Acknowledged. Each submitter sets extraArgs for its own message, so sizing the destination gas limit to the batch belongs to the caller, not the oracle contract.

RESEARCHER: Acknowledged.